WAS Experiment 09

Aim: Demonstration of File Inclusion Vulnerabilities

Learning Objective: To demonstrate file inclusion vulnerabilities in a controlled web application environment and understand how improper handling of user-controlled file paths can allow unauthorized files to be accessed or included by a web application.

Learning Objectives:
After completing this experiment, students will be able to:

- Understand the concept of File Inclusion vulnerabilities.
- Understand Local File Inclusion (LFI).
- Understand Remote File Inclusion (RFI).
- Identify vulnerable file parameters in a web application.
- Demonstrate LFI in a controlled laboratory environment.
- Understand the security impact of file inclusion vulnerabilities.
- Analyze requests using Burp Suite.
- Understand basic techniques for preventing file inclusion vulnerabilities.

Tools: Kali Linux, Apache Web Server, PHP, Terminal, Burp Suite, Web Browser-Firefox and Local or intentionally vulnerable web application.

Theory:

Introduction

File Inclusion is a web application vulnerability that occurs when an application dynamically includes a file based on user-controlled input without properly validating or restricting the supplied file path.
For example, an application may use a parameter to determine which page should be displayed:
http://localhost/index.php?page=home
If the application directly uses the value of page in a file inclusion operation without proper validation, an attacker may manipulate the parameter to access unintended files.
File Inclusion vulnerabilities are generally classified into:

1. Local File Inclusion (LFI)
2. Remote File Inclusion (RFI)

Local File Inclusion (LFI)

Local File Inclusion occurs when an attacker is able to make a vulnerable application include or access a file located on the same server.
A vulnerable application may contain logic similar to:
include($_GET['page']);
An attacker may manipulate the page parameter to reference an unintended local file.

Remote File Inclusion (RFI)

Remote File Inclusion occurs when a vulnerable application allows a file located on a remote server to be included.
This type of vulnerability is particularly dangerous because it may allow an attacker to introduce content from an external location.

Modern PHP configurations and secure application practices can significantly reduce the possibility of RFI.

Prevention Techniques

File Inclusion vulnerabilities can be prevented by:

- Avoiding direct use of user input in file inclusion functions.
- Using an allowlist of permitted files.
- Mapping user-controlled identifiers to predefined file names.
- Validating and canonicalizing file paths.
- Preventing directory traversal.
- Disabling unnecessary remote file inclusion functionality.
- Applying appropriate file-system permissions.
- Keeping server-side software securely configured.
- Following secure coding practices.

A safer approach is to use predefined identifiers rather than accepting arbitrary file paths.
For example:

```
$pages = [
   "home" => "home.php",
   "about" => "about.php"
];

include($pages[$_GET["page"]]);
```

This prevents the user from directly specifying an arbitrary file path.

Procedure:

Step 1: Start Kali Linux

Start Kali Linux and open the Terminal.

Launch Burp Suite:

burpsuite

Select Temporary Project and continue with the default configuration.

Expected Output:

Burp Suite starts successfully.

Step 2: Configure Firefox

Configure Firefox to use Burp Suite as the local proxy:

Proxy Address: 127.0.0.1

Proxy Port: 8080

Expected Output:

Firefox traffic is routed through Burp Suite.

Step 3: Start the Laboratory Web Application

Start the authorized local web application.

Ensure that the web server is running.

For example:

http://localhost/

Expected Output:

The laboratory web application loads successfully.

Step 4: Identify a File Parameter

Open the application and identify a parameter that controls which page or file is loaded.

For example:

http://localhost/index.php?page=home

Here:

page=home

is the parameter being tested.

Expected Output:

A file-related parameter is identified.

Demonstrating Local File Inclusion

Step 5: Test Normal File Inclusion

First, access the application using the normal parameter:

http://localhost/index.php?page=home

Observe the page displayed by the application.

Expected Output:

The application loads the requested page normally.

Step 6: Test for LFI

In the intentionally vulnerable laboratory application, test whether the page parameter accepts unintended local file paths.

A common laboratory test is:

http://localhost/index.php?page=../../../../etc/passwd

The exact number of directory traversal levels depends on the application's directory structure.

Expected Output:

If the application is vulnerable, the contents of the targeted local file may be displayed in the response.

This test should only be performed against a deliberately vulnerable local/lab application or another explicitly authorized target.

Step 7: Analyze the Request Using Burp Suite

Capture the request using:

Proxy → Intercept

A request may appear as:

GET /index.php?page=../../../../etc/passwd HTTP/1.1

Host: localhost

Analyze the request and response.

Expected Output:

The manipulated file parameter and server response can be observed in Burp Suite.

Demonstrating Remote File Inclusion

Step 8: Identify RFI Conditions

Remote File Inclusion depends on the application's configuration and whether remote file inclusion is permitted.

For a controlled laboratory application specifically configured to demonstrate RFI, a remote test file can be referenced through the vulnerable file parameter.

For example:

http://localhost/index.php?page=http://lab-server/test.txt

Expected Output:

If the laboratory application is configured to allow remote inclusion, the remote test content may be included.

Step 9: Analyze the Response

Examine the response generated by the application.

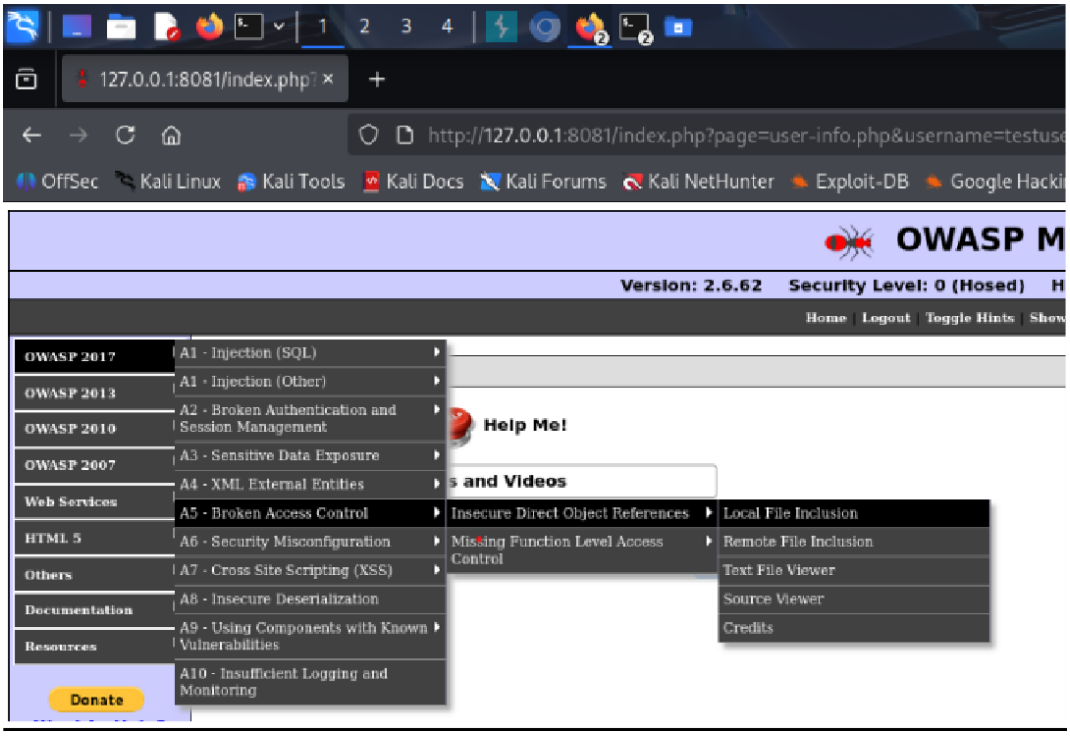
Observe whether the remote content is:

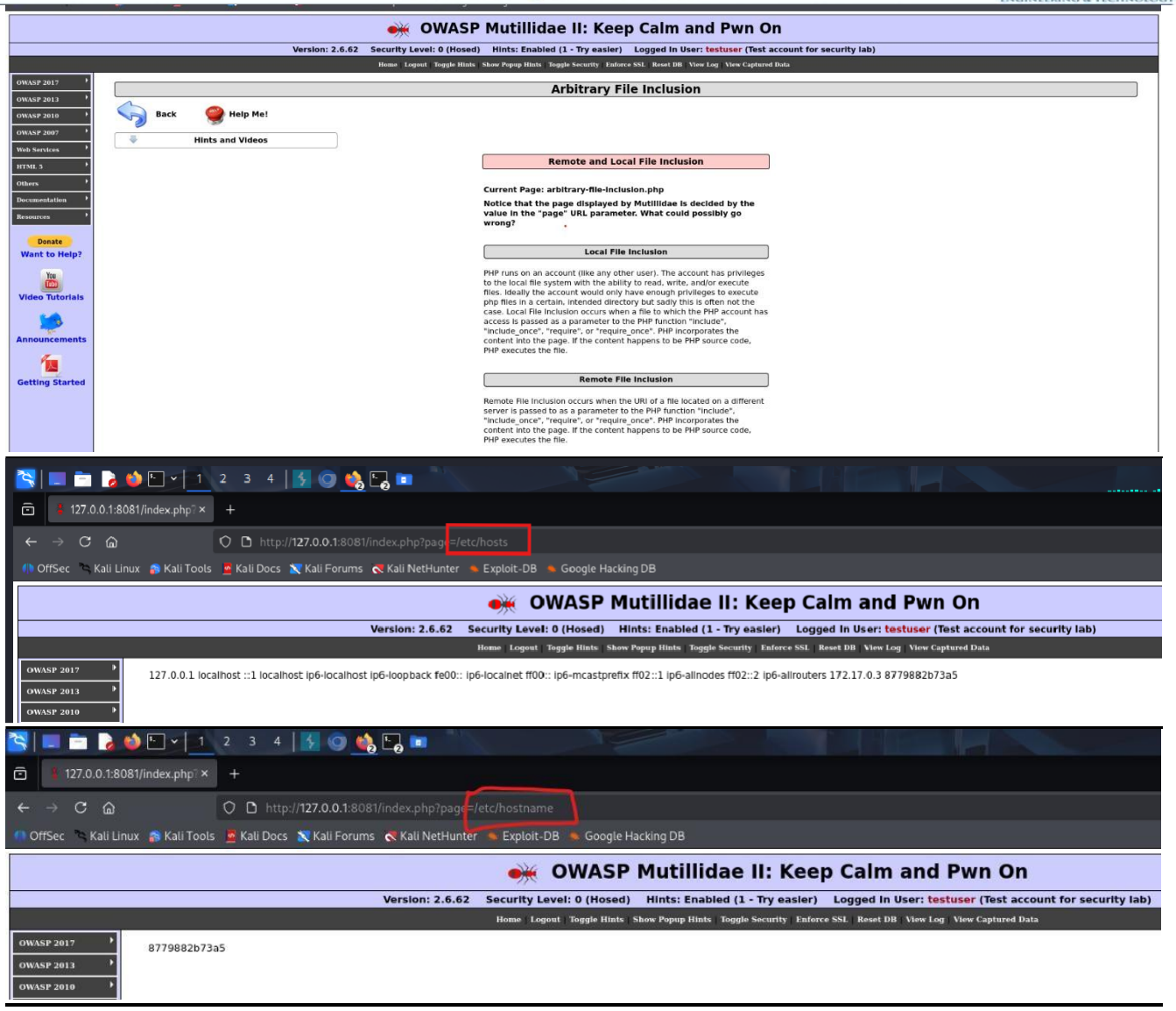
- Included
- Displayed
- Rejected
- Sanitized

Expected Output:

The application's handling of remote file references is identified.

Outputs:





Observation:

The following observations are made:

1. File Inclusion vulnerabilities can occur when applications trust user-controlled file parameters.
2. LFI involves files located on the local server.
3. RFI involves files located on a remote system.
4. Burp Suite can be used to intercept and modify file-related parameters.
5. Directory traversal may be used to test for LFI in vulnerable applications.
6. Proper input validation and file restrictions are important for preventing File Inclusion vulnerabilities.

Result and Discussion:

The experiment successfully demonstrated the concept of **File Inclusion vulnerabilities** in a controlled laboratory environment.

A file-related parameter was identified and tested using controlled input. Local File Inclusion was

demonstrated by attempting to reference a local file through a vulnerable parameter. The experiment also explained the conditions required for Remote File Inclusion and demonstrated how the application's response can be analyzed using Burp Suite.

File Inclusion vulnerabilities can expose sensitive information and, depending on the application's configuration and other security conditions, may have more serious consequences.

Conclusion:

This experiment demonstrated Local File Inclusion (LFI) and explained Remote File Inclusion (RFI) vulnerabilities in a controlled web application environment.

The experiment showed how improper handling of user-controlled file parameters can allow unintended files to be included or accessed. Burp Suite was used to inspect and manipulate HTTP requests during testing.

Proper input validation, allowlisting, secure file-path handling, and appropriate server configuration are essential for preventing File Inclusion vulnerabilities.

Viva Questions:

1. What is a File Inclusion vulnerability?
2. What is the difference between LFI and RFI?
3. What is Local File Inclusion?

For Faculty Use:

Correction Parameters	**Formative Assessment [40%]**	**Timely completion of Practical [40%]**	**Attendance / Learning Attitude [20%]**	
Marks Obtained				